

Figure 5: Web application with scalable storage network

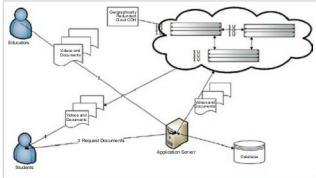


Figure 6: Web application with geo-redundant storage and a content delivery network

But the drawback of such stores is the SLA in retrieval since ‘Cold Storage’ is not accessible in real-time and only available on-demand. SLAs are usually in hours and vary with each cloud provider.

2. For redundant storage, one has to pay extra for redundancy.

Figure 6 shows how the Web application architecture will look at this stage.

Practice #6: Divide the application tiers further to scale only the components that need to be scaled.

A multi-tiered approach will help application components to have dedicated resources and can be configured in low-end servers which can be scaled-out/in, on demand.

With the help of further tiers, we are able to achieve the highest level of elasticity from the deployment architecture standpoint. However, managing a plethora of servers could become a nightmare and scaling them independently is the worst part. But, thankfully, with the evolution of the cloud and the elastic nature of the infrastructure, one need not worry about managing scale-out/in (which is mostly automatic).

Practice #7: The background jobs function in a multi-threaded environment, so one must ensure jobs are completed on time even with huge amounts of data to be processed.

There may be certain use cases that one could come across like processing orders, sending emails, sending notifications, etc. These include processes that cannot be processed in real-time, or those that can be processed only at a certain time (like sending newsletters to customers). This occurs mostly

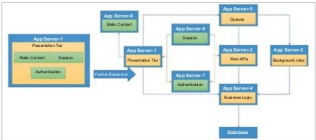


Figure 7: Application server tiers – each component is self-scalable

in monolithic applications cases, in which case, services or daemons are created to run on top of databases, picking one job at a time and processing it.

One can scale such operations using queues, wherein the jobs to be processed get added to the queue, and one or ‘n’ worker processes can execute the jobs in parallel. One can achieve scalability by increasing or decreasing the number of worker processes. The most popular queues in the open source world are RabbitMQ, ZeroMQ, Apache ActiveMQ, Kafka and BeanStalkD (see Table 3 for a comparative analysis). All of these work with most major languages and one can easily find the client libraries for them. There are other options available when it comes to message broker mechanisms, like ActiveMQ, IronMQ, Apache Kestrel, Apache QPID, etc.

Table 3

Technology	Pros	Cons
RabbitMQ	ACK and NACK, re-queuing and delay/scheduled messaging support Data persistence is built-in	Not easy to scale The central node adds latency if scaled
Kafka	Very high throughput Data persistence is default Supports online as well as offline processing	Transaction support not available but can be implemented
ZeroMQ	Persistence is default Routing is easy Easily scalable	No transaction support

Scalability is an integral part of the software design process and the application must be designed such that the application components can be de-coupled and can scale independently. The concept of scaling will also require one to design applications to have no footprint on the application server so that there can be quick allocation/de-allocation of servers. There are patterns available in each technology to achieve such goals and it is a matter of how and when to implement it. 

By: Manish Sharma and Brij Raj Singh

The authors are senior technical evangelists at Microsoft India.